IT PROF EL 4 - Advanced System Integration and Architecture

Semester Project, System Integration Documentation

INTEGRATING APPLICATIONS AND INTEGRATING THE ENTERPRISE

Submission Form

Partner 1: Raymond Christian A. Galanza	**Partner 2: Paz Girliamne Q. Sacabin**
Course/Section: BSIT-IV 80107	**Date: October 7, 2026**

Chosen App or Organization (semester project):	AI-Powered Campus Helpdesk and Ticket Routing System Using Multi-Label Text Classification

Part 1. Integration Map and Legacy Check

1A. Integration map (list at least 4 connections between systems)

From (system)	To (system)	Data that moves	How it moves today (direct link, file, API, or by hand)
Student Helpdesk Web App	AI Ticket Routing Service	Ticket subject, description, student ID, selected concern	API
AI Ticket Routing Service	Supabase (database)	Ticket record, predicted office labels, confidence, status	API
Student Helpdesk Web App	Supabase (database)	Student profile, submitted tickets, ticket status, feedback	API
Helpdesk System	Notification Service	Ticket number, assigned office, status update, feedback message	API
Student Helpdesk Web App	Campus Map Mode	Office location and navigation information	Direct Link

Which connections are point-to-point, and why can this become hard to manage as more systems are added? (2 to 3 sentences)

The Student Helpdesk Web App to the AI Ticket Routing Service, the AI Ticket Routing Service to the Supabase (database), and the Helpdesk System to the Notification Service are point-to-point connections because each directly communicates with one specific system. These links can become hard to manage as more systems are added because each new system may require several separate connections, increasing the number of interfaces that must be maintained, secured, and tested.

1B. Legacy check

Field	Entry
Older system, tool, or process the app still depends on	Manual ticket handling through student service assistants, telephone calls, email, and SMS
Why it is still used	These methods are still used because students and offices are already familiar with them and they can continue to receive concerns even when the new helpdesk system is not being used.
Main challenge in connecting it	The information is unstructured and may be spread across different channels, making it difficult to automatically classify, track, and synchronize ticket status.

Field	Entry
Approach to connect it (adapter or wrapper, middleware, staged replacement, data synchronization, or encapsulation)	Staged replacement
Why this approach fits	The new helpdesk can gradually handle common concerns and route them automatically while existing manual channels remain available during the transition. As users and offices adopt the system, the older manual process can be reduced and eventually used only as a fallback.

Part 2. Enterprise Integration: Shared Data

Shared data item (one data item used by two or more systems):	Campus Helpdesk Ticket

Shared data format (at least 5 fields)

Field name	Meaning	Example value
TicketID	Unique identifier for the helpdesk ticket	TCK-00125
StudentID	Identifier of the student who submitted the concern	2026-00123
Subject	Short title of the concern	Database 1 enrollment concern
Description	Detailed text of the student's concern	I need assistance with my Database 1 enrollment
AssignedOffices	Office or departments selected by the routing process	CCS Office; Registrar's Office
Status	Current state of the ticket	Routed

Systems that will use this shared format:	Student Helpdesk Web App, AI Ticket Routing Service, Supabase, Notification service, and Faculty/Staff Office Interface

Why is one shared format better than each system using its own format? (2 to 3 sentences)

One shared format gives all participating systems the same field names and meaning for a ticket. This reduces data conversion errors and makes it easier to add another office or service because the new system can use the same ticket structure instead of creating a separate format.

Part 3. Integration Strategy Choice

Field	Entry
Chosen strategy (point-to-point, API-led or service-oriented, message or event-driven, or middleware/ESB)	API-Led or Service Oriented
One advantage for this app	The Student Helpdesk Web App, AI routing service, office interface, and other future clients can use defined services to exchange ticket data without directly depending on each other's internal implementation.

Field	Entry
One disadvantage for this app	The services require clear interfaces, authentication, validation, and version management, which adds development and maintenance work for a small project team.
Why this strategy fits this app (2 to 3 sentences)	API-Led fits this app because several parts of the system need the same ticket data and routing functions. A defined service interface lets the student interface, office interface, database layer, and future clients communicate through consistent requests and responses.
One strategy not chosen, and why	Point-to-point, because direct connections would become difficult to maintain as more offices, notification channels, and campus services are integrated.

Part 4. Service Entry and Best Practices

4A. Service entry (one specific service, not the whole app)

Field	Entry
Service name	Ticket Routing Service
What it does	Receives a student's submitted concern, uses multi-label text classification to identify the appropriate office or offices, and returns the routing result.
Who uses it	Student Helpdesk Web App and the Faculty/Staff Office Interface
Input (what it needs)	TicketID, StudentID, Subject, description
Output (what it returns)	AssignedOffices, confidence score, routing status, and TicketID
Data format (XML or JSON) and why	JSON, because the web application and service can exchange structured ticket fields directly and consistently.
Who is responsible for it	The development team responsible for the Campus Helpdesk System
Security control (for example, an API key)	Authenticated requests using the application's access token/JWT, with request validation before processing ticket data.
Version (start at v1)	v1

4B. Best practices applied (2 XML or data format practices and 2 web service practices)

Type	Practice chosen	How it applies to this service (one sentence)
XML or data format practice 1	Use clear field names	The Ticket Routing Service uses names such as TicketID, StudentID, Subject, Description, and AssignedOffices so the web and office interfaces interpret the ticket data consistently.
XML or data format practice 2	Return clear, field-specific error messages	If a required field is missing or invalid, the service returns an error message that identifies the specific field, such as "Description is required."
Web service practice 1	Version the interface	The routing endpoint starts at v1 so future changes to the routing request or response can be introduced without unexpectedly breaking existing clients.

Type	Practice chosen	How it applies to this service (one sentence)
Web service practice 2	Validate requests	The service validates the StudentID, Subject, Description, and required request fields before running the ticket-routing process.